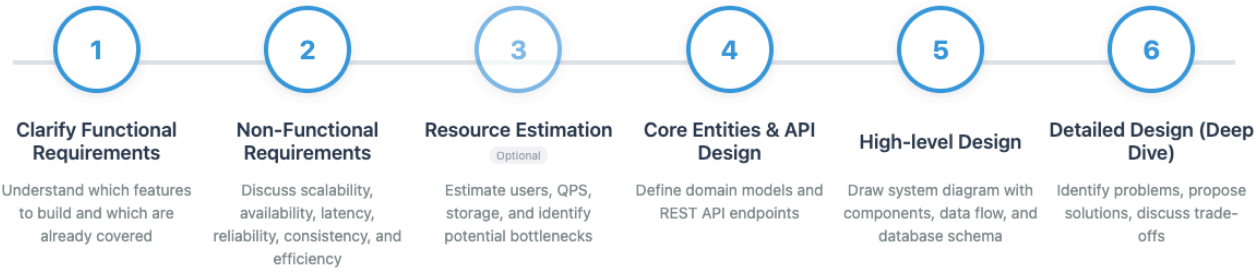
System Design Interview Step-by-step Guide

systemdesignschool.io/fundamentals/system-design-interview-template



Interview Template

Core 6:

- Scalability: Design the system to be able to handle the target number of users.
- Availability: Design the system to be operational when needed by using techniques like replication to avoid downtime.
- Latency: Identify the acceptable latency for the system. This varies depending on the application (e.g., Google Docs requires low latency, while Dropbox can handle longer latencies).
- Reliability: Ensure that the service consistently returns correct and expected results.
- Consistency: Address data consistency between services, ensure reading from different sources gives the same result, and decide where to save the data.
- Efficiency: Minimize redundant operations and optimize resource usage.

Special:

> Privacy: Consider GDPR compliance and other privacy requirements depending on the data being stored and processed (e.g., email, user information).

Put these requirements in the context of the problem being discussed.

3. Resource Estimation (optional)

Note that this step is often optional and the interviewer may just say "a lot of users" or "a lot of data", especially for product design questions such as a designing a social media platform or ride-sharing app. Resource estimation becomes important for infrastructure design questions such as designing a rate limiter or a caching system.

Ask:

- How many daily users does the system need to support (i.e. Daily Active Users aka DAU)?
- How long should the data be stored (i.e. data retention)?

Estimate:

- QPS (Queries per second):
- Ongoing connections (if applicable), e.g., for WebSocket-based applications like WhatsApp.
- Throughput (in GB/s), needed for video streaming or image hosting problems.
- Storage requirements based on data retention and object byte sizing using the number of strings.
- Identify potential bottlenecks in the system, such as DB or QPS limitations.

4. Core Entities and API Endpoint Design

Core entities are the core domain models that are required to implement the functional requirements. For example, if the system is a social media platform, the core entities might include users, posts, comments, and likes. These typically correspond to the database tables. It's relatively straightforward to identify core entities once we have the functional requirements- they are the **nouns** in the requirements.

APIs establish a contract between the service and the end user. Usually in this part we want to specify REST API request and response interface. Note that we only care about the APIs related to the functional requirements and avoid introducing irrelevant APIs. This step can also be done after Step 5 high level design if the interviewer considers this to be too detailed. These are the **verbs** in the requirements.

If the system necessitates bidirectional communication, don't forget to discuss technologies that facilitate this, such as WebSockets, or server-to-client messaging protocols like Server-Sent Events.

5. High-level Design (Design Diagram)

Draw a high-level system diagram that includes:

- Main components of the systems and how they interact with each other.
- Data flow: Show how data moves through the system.
- Traffic pattern: If relevant to the problem, show how the system handles different types of traffic (e.g., push vs. pull for Twitter).
- Database model/schema, and which fields to index if necessary.

Make sure every feature in the functional requirements is covered in the design. However, don’t dive deep into the details too early at this point.

Note that, in most cases, when we're sketching out system design, we primarily focus on the key components that are specific to the unique problems the system is aiming to solve. Common elements like authentication, which are parts of virtually all systems, are often left out

of these initial diagrams.

It's also essential to bear in mind that during the initial stages of system design, it's not typical to delve into the particulars of specific technologies. Over-emphasizing buzzwords or trendy tech stacks can be a significant misstep, as interviewers may perceive it as an attempt to cover up a lack of understanding of the fundamental concepts.

6. Detailed Design (Deep Dive)

The goal of this section is to identify the parts of the components in the system that could lead to (mostly scalability) problems and discuss how to handle them, possibly by modifying the design. This section is more open-ended and relies on interaction with the interviewer. Non-functional requirements play a key role in making design decisions here.

Once we identify a problem, we should propose ideally multiple solutions and discuss their trade-offs. The trade-offs are usually along the lines of balancing scalability, latency and consistency.

How System Design School's Questions are Designed

As you can see, the interview format is quite open-ended. In System Design School's question format, we have to introduce some constraints to the problems to make the answers more directed so we can provide more standardized answers and give you feedback. Therefore, Step 1 and 2 are defined for you in the problem description, and Steps 3-6 are for you to complete.

Staff Engineer Interviews

Note that if you are interviewing for a staff level position (Google L6+, Amazon Principle+ etc, see level comparison chart), the template may be very different. At this level, interviews are less about cookie-cutter questions like back-of-the-envelope calculation or API design. Such calculations can be tedious and detract from more meaningful tasks. While the ability to make rough resource estimates using basic arithmetic is useful, it is generally only a bonus for senior roles and rarely a deciding factor. Therefore, it's best to avoid it unless specifically requested.

Instead, interviewers might ask you to design hypothetical scenarios aiming to make the system infinitely scalable or handle fault tolerance in even unrealistic situations.

The focus at the staff level is on identifying difficult challenges within the requirements and designing a system to address them. The interview process can vary significantly depending on the interviewer, but it generally follows this flow:

- **Clarify the problem requirements**: Ensure you fully understand the problem by asking detailed questions.
- **Identify core challenges**: Determine the main difficulties that need to be addressed.

- **Create a solution to solve the challenges**: Propose a design that addresses these challenges using only simple components. Note that the "simple components" part is important. The design may require experience to come up with, but once conceived the design itself should be easily understood by junior engineers so they can start coding.
- **Work with the interviewer on deep dives**: Explore specific parts of your design in detail.

And of course, don't forget to be a nice person and show off your communication skills. At this level, people really want to know you are easy to work with. Demonstrating strong interpersonal skills and being a pleasant person to collaborate with can make a big difference in your evaluation.

Start-up Engineer Interviews

Early-stage startups hire very differently compared to large tech companies, and their interview processes reflect this difference. The primary goal of early-stage startups is to build products quickly with limited resources. They typically seek candidates who can start contributing immediately. As a result, their interview process focuses on practical skills and real-world applications rather than theoretical scalability challenges and edge cases.

Startups usually want you to design simple, usable systems that can be built and deployed quickly. Unlike in larger companies, where mentioning specific technologies in system design interviews is often discouraged, startups appreciate it when candidates understand their tech stack and use relevant components in their designs. This demonstrates that you are familiar with their technology and would require less time to onboard.

The interview process at a startup generally includes:

- **Understanding the tech stack**: Familiarize yourself with the technologies the startup uses.
- **Designing practical solutions**: Focus on creating simple, effective systems that can be quickly implemented.
- **Showing immediate value**: Demonstrate how you can start contributing quickly.

By tailoring your approach to the specific needs and context of the startup, you can better showcase your ability to meet their immediate and practical needs.